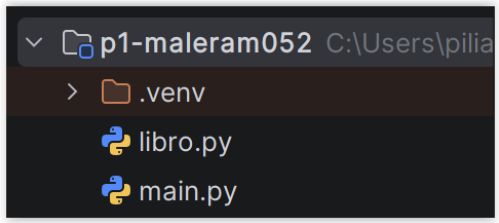
Práctica 1. Gestión básica de una biblioteca.

Crear un proyecto llamado **p1-<usuariomoodle>**:

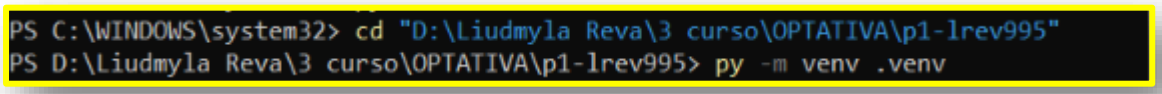


Aunque esta práctica solo utiliza la biblioteca estándar de Python y no necesita instalar dependencias externas, vamos a crear un entorno virtual.

Crear el **entorno:**

```
python -m venv .venv
```

En esta captura creé el entorno virtual .venv dentro de la carpeta de mi proyecto usando el comando py -m venv .venv para aislar las dependencias del trabajo.



Activarlo en PowerShell:

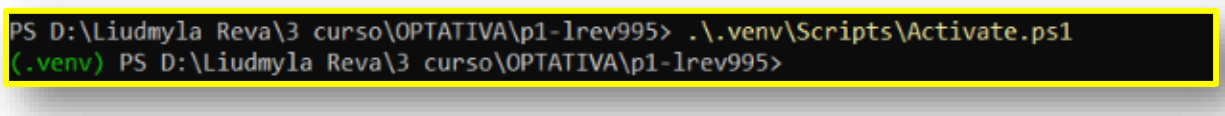
```
.\.venv\Scripts\Activate.ps1
```



Para desactivarlo:

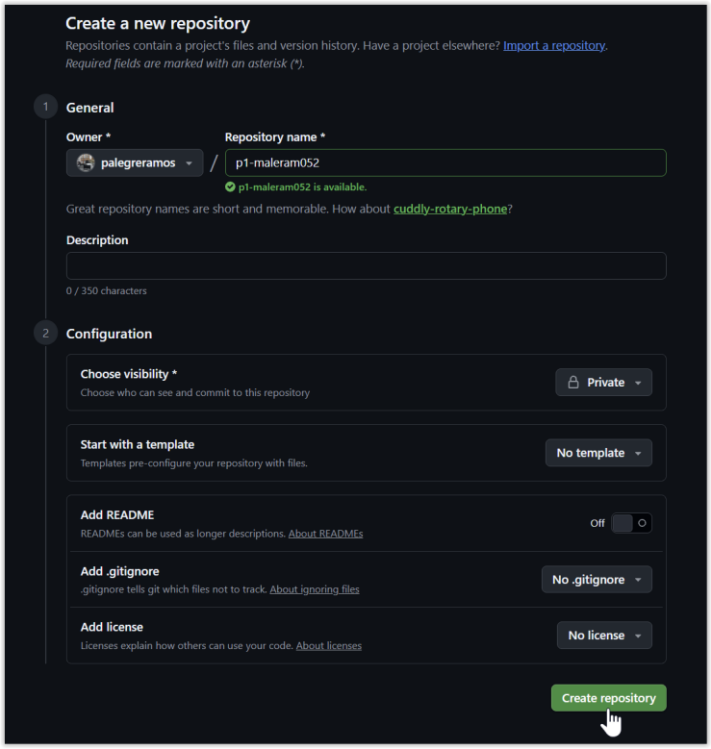
```
Deactivate
```

Aquí ejecuté el script de activación .\.venv\Scripts\Activate.ps1. Como se observa en la consola, aparece el prefijo (.venv) al principio de la línea, indicando que el entorno virtual ya está activo.

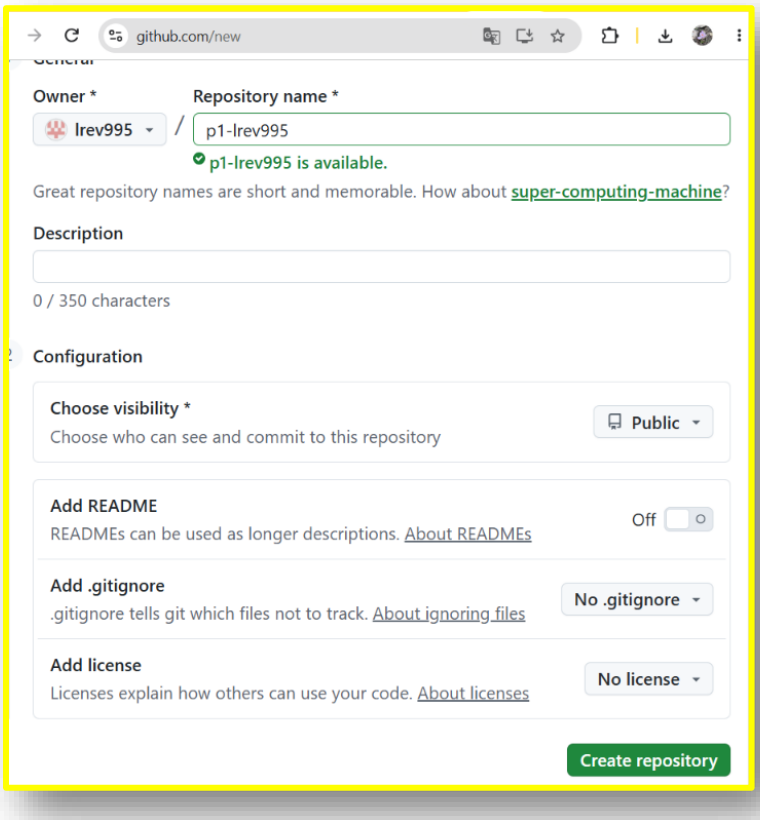


Crear repositorio en **GitHub:**

Crear un repositorio llamado **p1-<usuario moodle>** sin añadir ni README, ni .gitignore, ni licencia.



Aquí he creado repositorio en GitHub llamando ***p1-lrev995***



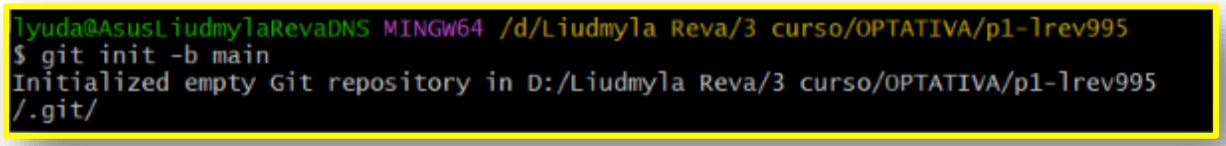
Inicializa el repositorio local con **Git**:

`git init`

Si quieres asegurarte que su rama principal sea main:

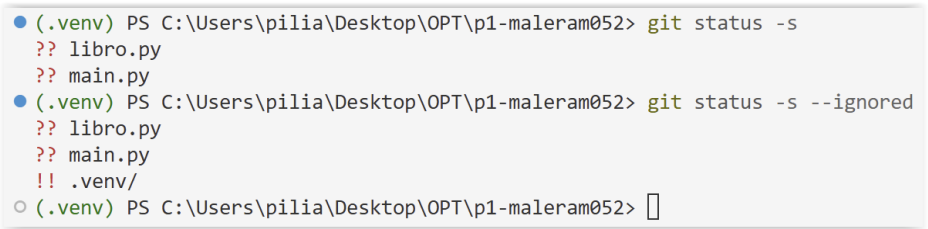
`git init -b main`

Aquí inicialicé el repositorio local de Git en la carpeta del proyecto usando el comando git init -b main para asegurarme de que la rama principal se llame directamente main.



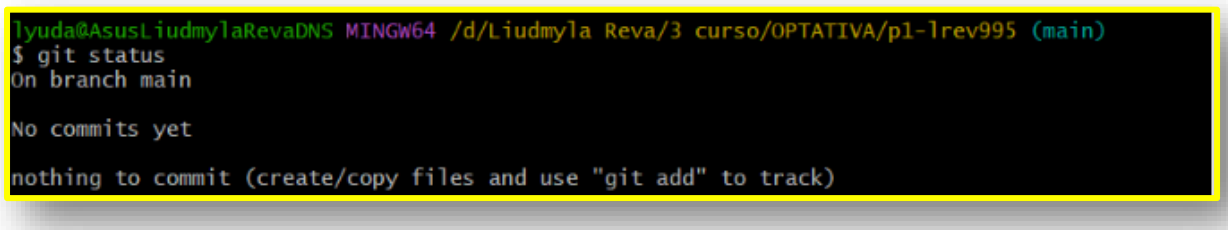
Comprueba el status:

`git status`



?? indica que todavía no tiene seguimiento y **!!** que está siendo ignorado.
Eso es así porque .venv trae su propio gitignore, pero descargamos el nuestro.

Ejecuté el comando git status y verifiqué que estoy correctamente ubicada en la rama main. A diferencia de la captura del enunciado donde ya aparecen los archivos libro.py y main.py con ??, en mi caso el repositorio está completamente limpio (nothing to commit), ya que aún no hemos recibido las instrucciones para crear dichos archivos ni su contenido, lo cual haremos a continuación en el Ejercicio 1.



Crear .gitignore:

Descargar el fichero en la raíz del proyecto y renombrarlo como **.gitignore**

https://github.com/github/gitignore/blob/main/Python.gitignore

Desde la carpeta local, vincular el repositorio local con el repositorio remoto:

```
git remote add origin https://github.com/<USUARIO>/p1-<moodle>.git
```

git config list (verificar rama, nombre usuario, email y remoto)

Usaremos un único repositorio, una rama main y etiquetas v1, v2, ..., v8 por cada ejercicio.

Ejercicio 1. Crear una clase Libro en **`libro.py`**

En **`libro.py`**

```
class Libro:
    pass
```

pass instrucción que no realiza ninguna operación, pero se utiliza porque Python necesita el bloque de código, pero todavía no lo hemos implementado

En **`main.py`**

importa la clase, crea un objeto y muestra tanto el objeto como su tipo

```
from libro import Libro

libro1 = Libro()

print(libro1)
print(type(libro1))
```

En **`Git`**

Añadimos los archivos que se han modificado, comprobamos el estado, consultamos los archivos con seguimiento, creamos el commit, la primera versión, subimos la rama principal y la etiqueta:

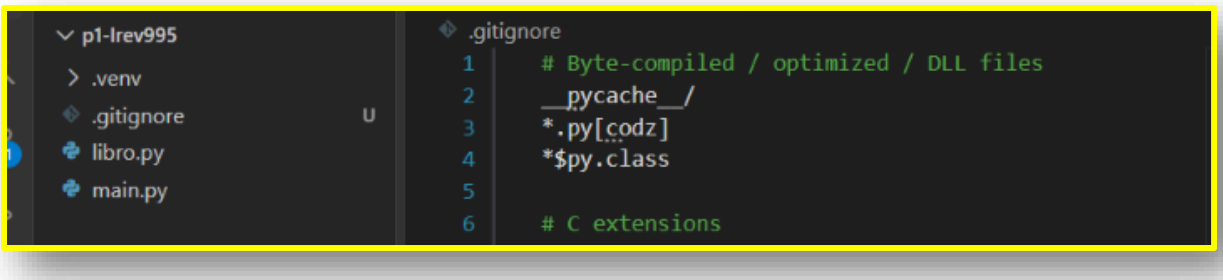
```
git add .
git status –s
```

Comprobé el estado del proyecto ejecutando primero git status -s para ver los archivos pendientes (??), y luego git status -s --ignored para verificar los elementos excluidos. En la consola se aprecia con !! que la carpeta .venv/ ya está siendo ignorada correctamente gracias a la configuración por defecto de Python.

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git status -s
?? libro.py
?? main.py

lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git status -s --ignored
?? libro.py
?? main.py
!! .venv/
```

Descargué el archivo Python.gitignore desde el repositorio oficial indicado en las instrucciones, lo renombré a .gitignore en la raíz de mi proyecto y vinculé el repositorio local con GitHub usando git remote add origin.




```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git remote add origin https://github.com/lrev995/p1-lrev995.git
```

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
```

git ls-files

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git status -s
?? .gitignore

lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git ls-files
libro.py
main.py
```

git commit -m "Ejercicio 1"
git tag v1

Añadí los archivos al área de preparación con git add ., verifiqué el estado y creé el primer commit con la etiqueta v1 para la estructura básica de la clase Libro.
Reasigné la etiqueta v1 al último commit usando git tag -f v1 para asegurarme de que quedara vinculada correctamente al estado actual del Ejercicio 1.

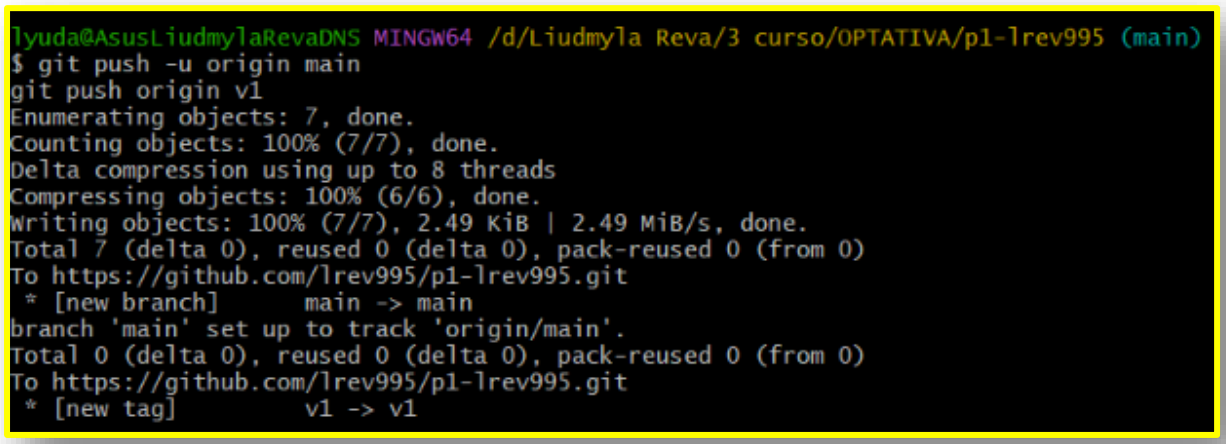
```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git commit -m "Ejercicio 1"
git tag v1
[main ac56b85] Ejercicio 1
 1 file changed, 223 insertions(+)
 create mode 100644 .gitignore
fatal: tag 'v1' already exists

lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git tag -f v1
Updated tag 'v1' (was fbe8e2f)
```

git push -u origin main
git push origin v1

-u (upstream): establece la conexión de seguimiento entre la rama local main y la rama remota origin/main. Así, en las siguientes ocasiones, podemos utilizar simplemente git push y git pull, sin indicar el repositorio y la rama. Le indica a Git cuál es la rama remota asociada a nuestra rama local, y así no hay que especificarla en cada push o pull.

Subí por primera vez la rama principal al repositorio remoto con `git push -u origin main` para vincular el seguimiento local y remoto. Además, publiqué la primera etiqueta con `git push origin v1`.



Ejercicio 2. Añadir constructor y atributos de instancia

En **libro.py**

```
class Libro:
    def __init__(self, titulo, autor, anio):
        self.titulo = titulo
        self.autor = autor
        self.anio = anio
        self.disponible = True
```

En **main.py**

https://gist.github.com/palegreramos/3d0f051491f0d83004b8295d90398f47

```
from libro import Libro

def main():
    libros = [
        Libro("La colmena", "Camilo José Cela", 1951),
        Libro("Cien años de soledad", "Gabriel García Márquez", 1967),
        Libro("La casa de Bernarda Alba", "Federico García Lorca", 1936),
        Libro("Nada", "Carmen Laforet", 1945),
        Libro("La familia de Pascual Duarte", "Camilo José Cela", 1942)
    ]

    for libro in libros:
        print(libro.titulo, libro.autor, libro.anio)

if __name__ == "__main__":
```

```
    main()
```

En Git

Consultamos las modificaciones:

```
git status -s
git diff
```

Ejecuté `git status -s` para verificar los archivos modificados (`M`) y utilicé `git diff` para revisar en detalle las diferencias de código: el nuevo constructor `__init__` en `libro.py` y la función `main()` con la lista de libros en `main.py`.

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git status -s
git diff
 M libro.py
 M main.py
warning: in the working copy of 'libro.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
diff --git a/libro.py b/libro.py
index d8213e2..1f3a945 100644
--- a/libro.py
+++ b/libro.py
@@ -1,2 +1,7 @@
 class Libro:
-    pass
\ No newline at end of file
+    def __init__(self, titulo, autor, anio):
+        self.titulo = titulo
+        self.autor = autor
+        self.anio = anio
+        self.disponible = True
+
diff --git a/main.py b/main.py
index 8d8480f..b6f7c44 100644
--- a/main.py
+++ b/main.py
@@ -1,6 +1,15 @@
 from libro import Libro

-libro1 = Libro()
+def main():
+    libros = [
+        Libro("La colmena", "Camilo José Cela", 1951),
+        Libro("Cien años de soledad", "Gabriel García Márquez", 1967),
+        Libro("La casa de Bernarda Alba", "Federico García Lorca", 1936),
+        Libro("Nada", "Carmen Laforet", 1945),
+        Libro("La familia de Pascual Duarte", "Camilo José Cela", 1942)
+    ]
+    for libro in libros:
+        print(libro.titulo, libro.autor, libro.anio)

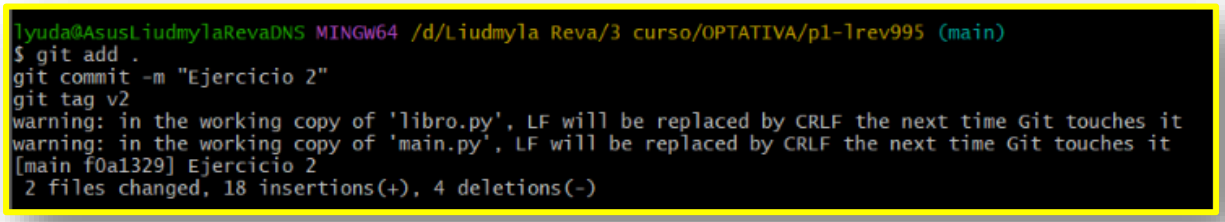
-print(libro1)
-print(type(libro1))
\ No newline at end of file
+if __name__ == "__main__":
+    main()
\ No newline at end of file
```

Creamos y subimos la nueva versión:

```
git add .
git commit -m "Ejercicio 2"
```

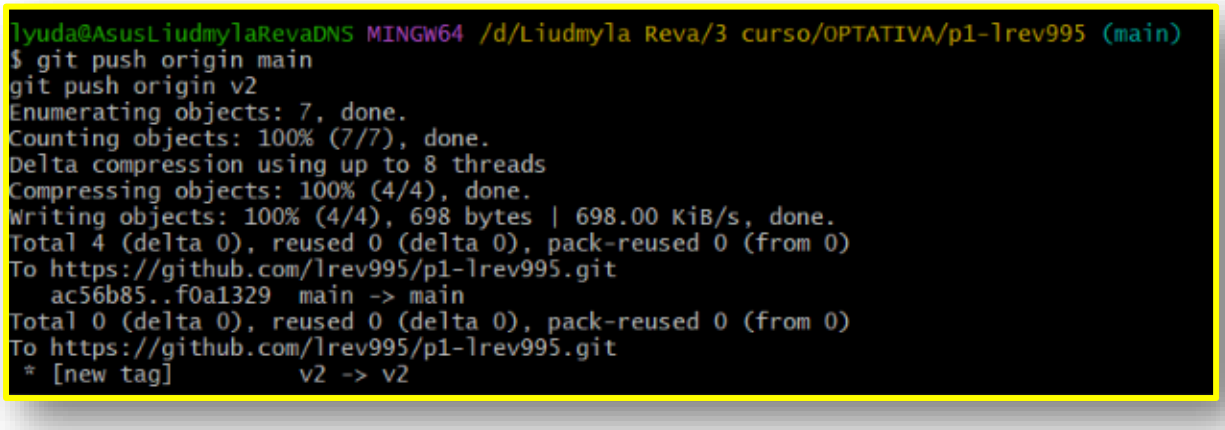
```
git tag v2
```

Añadí todos los cambios del Ejercicio 2 al área de preparación con `git add .`, registré el commit correspondiente mediante `git commit -m "Ejercicio 2"` y le asigné la etiqueta `v2` para dejar guardada esta versión en el repositorio.




```
git push origin main
git push origin v2
```

Subí los cambios del Ejercicio 2 a la rama remota con `git push origin main` y envié la etiqueta `v2` a GitHub mediante `git push origin v2` para mantener sincronizado el repositorio remoto con mis progresos locales.



Ejercicio 3. Añadir métodos de instancia

En **`libro.py`**

```
def prestar(self):
    if not self.disponible:
```

```
        raise ValueError("El libro ya está prestado")

    self.disponible = False


def devolver(self):
    if self.disponible:
        raise ValueError("El libro ya está disponible")

    self.disponible = True
```

En **main.py**

```
def intentar_prestar(libro):
    try:
        libro.prestar()
        print(f"Préstamo realizado: {libro.titulo}")

    except ValueError as error:
        print(f"Préstamo rechazado: {error}")


def intentar_devolver(libro):
    try:
        libro.devolver()
        print(f"Devolución realizada: {libro.titulo}")

    except ValueError as error:
        print(f"Devolución rechazada: {error}")
```

En **main()**

```
    libro = libros[1]
```

```
    try:
        libro.prestar()
        libro.prestar()
    except  ValueError as error:
        print(error)
```

```
    try:
        libro.devolver()
        libro.devolver()
    except  ValueError as error:
        print(error)
```

En Git: guardar y publicar v3

```
git status -s
git diff
```

Verifiqué las modificaciones con `git status -s` y `git diff`, confirmando la incorporación de los métodos `prestar()` y `devolver()` en `libro.py` junto con sus respectivas pruebas de captura de excepciones `ValueError` en `main.py`.

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git status -s
git diff
 M libro.py
 M main.py
warning: in the working copy of 'libro.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
diff --git a/libro.py b/libro.py
index 1f3a945..310b47a 100644
--- a/libro.py
+++ b/libro.py
@@ -5,3 +5,12 @@ class Libro:
         self.anio = anio
         self.disponible = True

+    def prestar(self):
+        if not self.disponible:
+            raise ValueError("El libro ya está prestado")
+        self.disponible = False
+
+    def devolver(self):
+        if self.disponible:
+            raise ValueError("El libro ya está disponible")
+        self.disponible = True
\ No newline at end of file
diff --git a/main.py b/main.py
index b6f7c44..09de975 100644
--- a/main.py
+++ b/main.py
@@ -1,5 +1,19 @@
 from libro import Libro

+def intentar_prestar(libro):
+    try:
+        libro.prestar()
+        print(f"Préstamo realizado: {libro.titulo}")
+    except ValueError as error:
+        print(f"Préstamo rechazado: {error}")
+
+def intentar_devolver(libro):
```

```
git add .
git commit -m "Ejercicio 3"
git tag v3
```

Preparé los cambios del Ejercicio 3 con `git add .`, guardé el commit de la versión `Ejercicio 3` y creé la etiqueta `v3` para marcar este hito en la historia del repositorio.

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git add .
git commit -m "Ejercicio 3"
git tag v3
warning: in the working copy of 'libro.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
[main 4838e2a] Ejercicio 3
 2 files changed, 29 insertions(+), 2 deletions(-)
```

```
git push origin main
git push origin v3
```

Subí los cambios del Ejercicio 3 a la rama remota con `git push origin main` y envié la etiqueta `v3` a GitHub mediante `git push origin v3` para mantener sincronizado el repositorio remoto con mis progresos locales.

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git push origin main
git push origin v3
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 892 bytes | 892.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/lrev995/p1-lrev995.git
   f0a1329..4838e2a  main -> main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/lrev995/p1-lrev995.git
 * [new tag]         v3 -> v3
```

Ejercicio 4. **Atributos de clase**

En **libro.py**

```
class Libro:
    total_libros = 0

    def __init__(self, titulo, autor, anio):
        self.titulo = titulo
        self.autor = autor
        self.anio = anio
        self.disponible = True

        Libro.total_libros += 1
```

En **main.py**

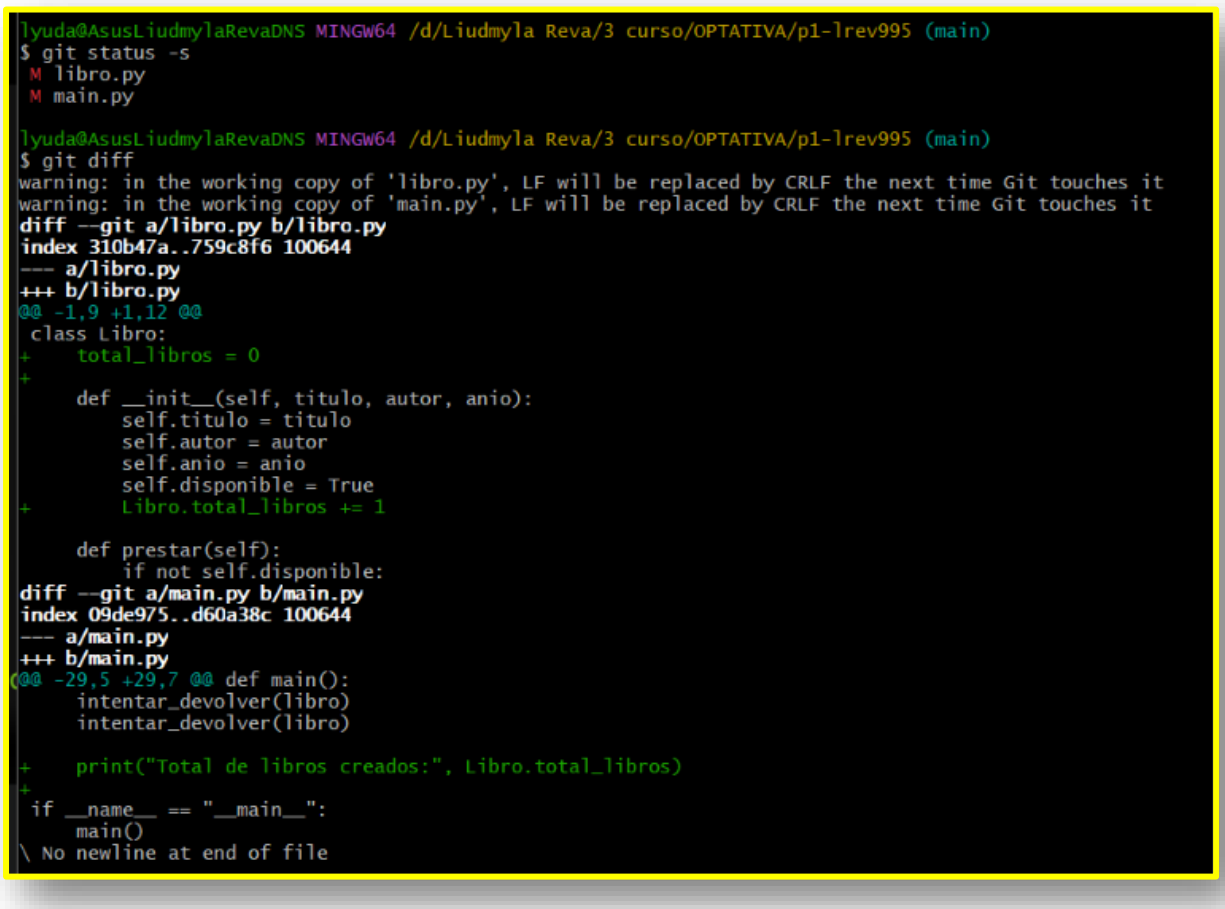
```
print("Total de libros creados:", Libro.total_libros)
```

En Git: guardar y publicar v4

```
git status -s
git diff
```

Ejecuté git status -s para comprobar el estado de los archivos del proyecto, verificando que tanto libro.py como main.py figuran como modificados (M) tras añadir el atributo de clase total_libros.

Utilicé git diff para revisar en detalle los cambios introducidos: la definición de la variable de clase total_libros = 0 y su incremento en el constructor dentro de libro.py, así como la impresión del total acumulado en main.py.



```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git status -s
 M libro.py
 M main.py

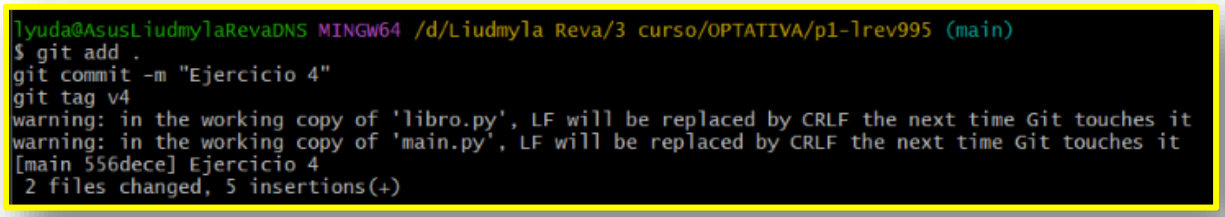
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git diff
warning: in the working copy of 'libro.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
diff --git a/libro.py b/libro.py
index 310b47a..759c8f6 100644
--- a/libro.py
+++ b/libro.py
@@ -1,9 +1,12 @@
 class Libro:
+    total_libros = 0
+
     def __init__(self, titulo, autor, anio):
         self.titulo = titulo
         self.autor = autor
         self.anio = anio
         self.disponible = True
+        Libro.total_libros += 1

     def prestar(self):
         if not self.disponible:
diff --git a/main.py b/main.py
index 09de975..d60a38c 100644
--- a/main.py
+++ b/main.py
@@ -29,5 +29,7 @@ def main():
     intentar_devolver(libro)
     intentar_devolver(libro)

+    print("Total de libros creados:", Libro.total_libros)
+
 if __name__ == "__main__":
     main()
\ No newline at end of file
```

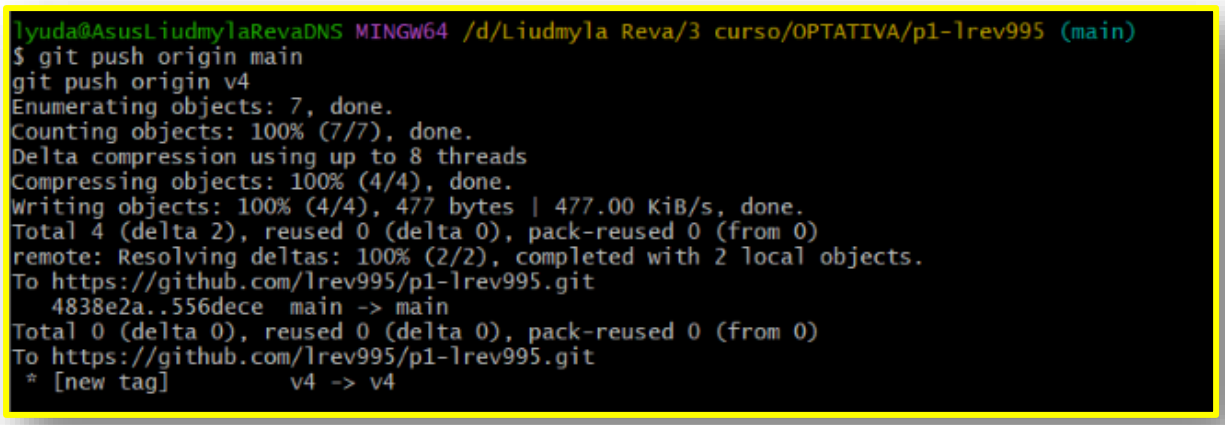
```
git add .
git commit -m "Ejercicio 4"
git tag v4
```

Preparé los cambios del Ejercicio 4 con `git add .`, registré el commit correspondiente al contador de libros y le asigné la etiqueta `v4` para dejar guardada esta versión en el repositorio.




```
git push origin main
git push origin v4
```

Subí los cambios del Ejercicio 4 a la rama remota con `git push origin main` y envié la etiqueta `v4` a GitHub mediante `git push origin v4` para mantener sincronizado el repositorio remoto con mis progresos locales.



Ejercicio 5. Propiedades: el decorador **`@property`** actúa como un getter para leer el valor de un atributo, mientras que **`@nombre_del_atributo.setter`** permite definir un método para modificarlo de forma controlada.

Por ejemplo, si escribimos:

```
libros[0].titulo = ""
```

Lo acepta.

Para que vuelva a tener título:

```
libros[0].titulo = "La colmena"
```

Si queremos impedir que un libro tenga un título vacío, añadimos:
@property permite leer el valor y **@titulo.setter** permite controlar su modificación.
En **libro.py**

```
    @property
    def titulo(self):
        return self._titulo

    @titulo.setter
    def titulo(self, nuevo_titulo):
        if not isinstance(nuevo_titulo, str):
            raise TypeError("El título debe ser texto")

        if not nuevo_titulo.strip():
            raise ValueError("El título no puede estar vacío")

        self._titulo = nuevo_titulo.strip()
```

Por ejemplo:
En **main.py**

```
try:
    nuevo_libro = Libro("", "Miguel de Unamuno", 1914)
    libros.append(nuevo_libro)
except ValueError as error:
    print(f"Error al crear el libro: {error}")
```

En Git: guardar y publicar v5

```
git status -s
git diff
```

Comprobé el estado del repositorio con `git status -s`, confirmando que los archivos `libro.py` y `main.py` aparecen marcados como modificados (`M`) tras implementar la validación del título del libro.

Revisé las diferencias de código con `git diff`: en `libro.py` añadí la propiedad `@property` y el `@setter` para validar que el título no esté vacío ni sea un tipo incorrecto, y en `main.py` agregué la captura de la excepción `ValueError` al intentar instanciar un libro sin nombre.

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git status -s
git diff
 M libro.py
 M main.py
warning: in the working copy of 'libro.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
diff --git a/libro.py b/libro.py
index 759c8f6..ed7bef0 100644
--- a/libro.py
+++ b/libro.py
@@ -8,6 +8,18 @@ class Libro:
         self.disponible = True
         Libro.total_libros += 1

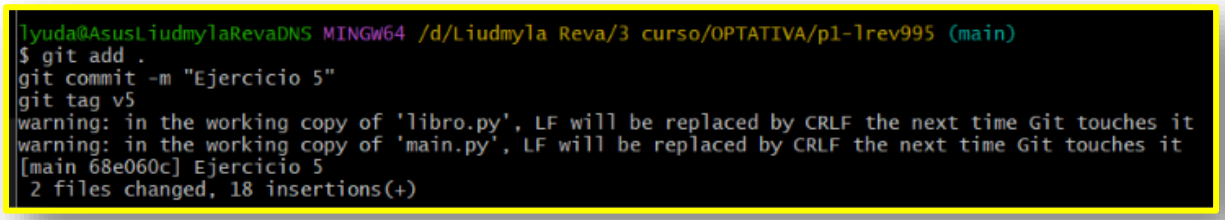
+    @property
+    def titulo(self):
+        return self._titulo
+
+    @titulo.setter
+    def titulo(self, nuevo_titulo):
+        if not isinstance(nuevo_titulo, str):
+            raise TypeError("El título debe ser texto")
+        if not nuevo_titulo.strip():
+            raise ValueError("El titulo no puede estar vacio")
+        self._titulo = nuevo_titulo.strip()
+
     def prestar(self):
         if not self.disponible:
             raise ValueError("El libro ya está prestado")
diff --git a/main.py b/main.py
index d60a38c..e34c2d0 100644
--- a/main.py
+++ b/main.py
@@ -31,5 +31,11 @@ def main():

     print("Total de libros creados:", Libro.total_libros)

+    try:
+        nuevo_libro = Libro("", "Miguel de Unamuno", 1914)
+        libros.append(nuevo_libro)
+    except ValueError as error:
+        print(f"Error al crear el libro: {error}")
+
 if __name__ == "__main__":
     main()
\ No newline at end of file
```

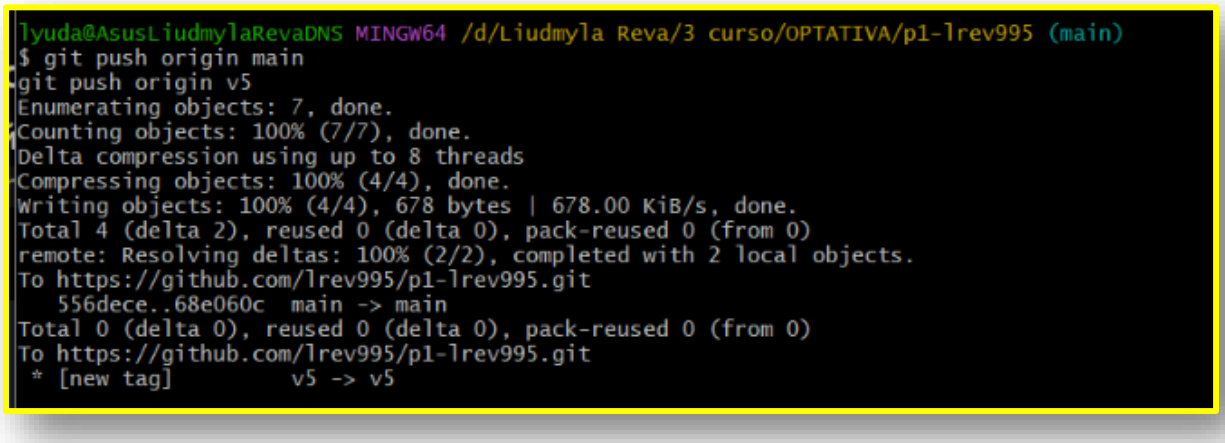
```
git add .
git commit -m "Ejercicio 5"
git tag v5
```

Añadí las validaciones del título al área de preparación con `git add .`, guardé el commit del Ejercicio 5 y asigné la etiqueta `v5` para registrar esta versión en el control de código.




```
git push origin main
git push origin v5
```

Subí los cambios del Ejercicio 5 a la rama remota con `git push origin main` y publiqué la etiqueta `v5` en GitHub mediante `git push origin v5`, confirmando que el nuevo tag ya está disponible en el repositorio remoto.



Ejercicio 6. `Propiedad calculada`

Una propiedad calculada permite obtener información a partir de los atributos de un objeto sin almacenar un valor adicional. En este caso, es_antiguo se calcula utilizando anio, por lo que su resultado se actualiza automáticamente cuando cambia el año de publicación.

En **libro.py**

```
    @property
    def es_antiguo(self):
```

```
        return self.anio < 1950
```

En **main.py**

```
    for libro in libros:
        print(libro.titulo, libro.es_antiguo)

   reciente = Libro("Ejemplo reciente", "Autor de ejemplo", 2022)

    print(reciente.es_antiguo)
```

En Git: guardar y publicar v6

```
git status -s
git diff
```

Ejecuté `git status -s` para comprobar el estado de los ficheros tras programar la propiedad `es_antiguo`, verificando que `libro.py` y `main.py` aparecen como modificados (`M`).

Revisé las modificaciones en detalle con `git diff`: en `libro.py` definí la propiedad calculada `es_antiguo` que evalúa si el año es anterior a 1950, y en `main.py` añadí la prueba con un libro reciente para validar su resultado.

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$
git status -s
git diff
 M libro.py
 M main.py
warning: in the working copy of 'libro.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
diff --git a/libro.py b/libro.py
index ed7bef0..bdc7548 100644
--- a/libro.py
+++ b/libro.py
@@ -28,4 +28,8 @@ class Libro:
     def devolver(self):
         if self.disponible:
             raise ValueError("El libro ya está disponible")
-        self.disponible = True
\ No newline at end of file
+        self.disponible = True
+
+    @property
+    def es_antiguo(self):
+        return self.anio < 1950
\ No newline at end of file
diff --git a/main.py b/main.py
index e34c2d0..d75444c 100644
--- a/main.py
+++ b/main.py
@@ -37,5 +37,8 @@ def main():
     except ValueError as error:
         print(f"Error al crear el libro: {error}")

+    reciente = Libro("Ejemplo reciente", "Autor de ejemplo", 2022)
+    print("¿Es antiguo el libro reciente?:", reciente.es_antiguo)
+
 if __name__ == "__main__":
     main()
\ No newline at end of file
```

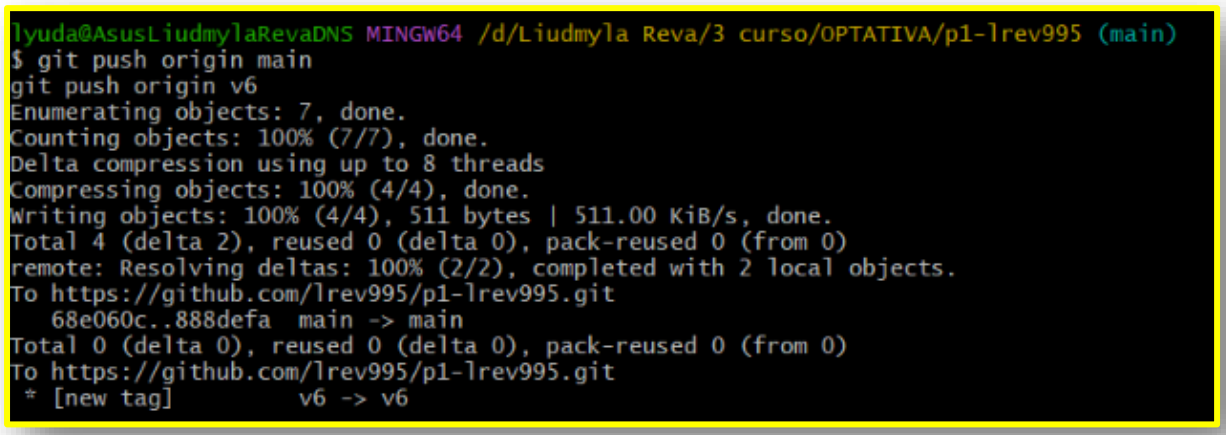
```
git add .
git commit -m "Ejercicio 6"
git tag v6
```

Añadí los cambios de la propiedad `es_antiguo` al área de preparación con `git add .`, registré el commit correspondiente al Ejercicio 6 (`888defa`) y le asigné la etiqueta `v6` para guardar esta versión en el historial.

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git add .
git commit -m "Ejercicio 6"
git tag v6
warning: in the working copy of 'libro.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
[main 888defa] Ejercicio 6
 2 files changed, 8 insertions(+), 1 deletion(-)
```

```
git push origin main
git push origin v6
```

Subí los cambios del Ejercicio 6 a la rama remota con `git push origin main` y publiqué la etiqueta `v6` en GitHub mediante `git push origin v6`, dejando sincronizado el estado del proyecto.



Ejercicio 7. **Métodos especiales `__str__()` y `__eq()__`**

En **libro.py**

```
    def __str__(self):
        estado = "Disponible" if self.disponible else "Prestado"
        return f"{self.titulo} - {self.autor} ({self.anio}) [{estado}]"

    def __eq__(self, otro):
        if not isinstance(otro, Libro):
            return NotImplemented

        return (
            self.titulo == otro.titulo
            and self.autor == otro.autor
            and self.anio == otro.anio
    )
```

En **main.py**

```
    for libro in libros:
        print(libro)
    otro_libro = Libro("Nada", "Carmen Laforet", 1945)

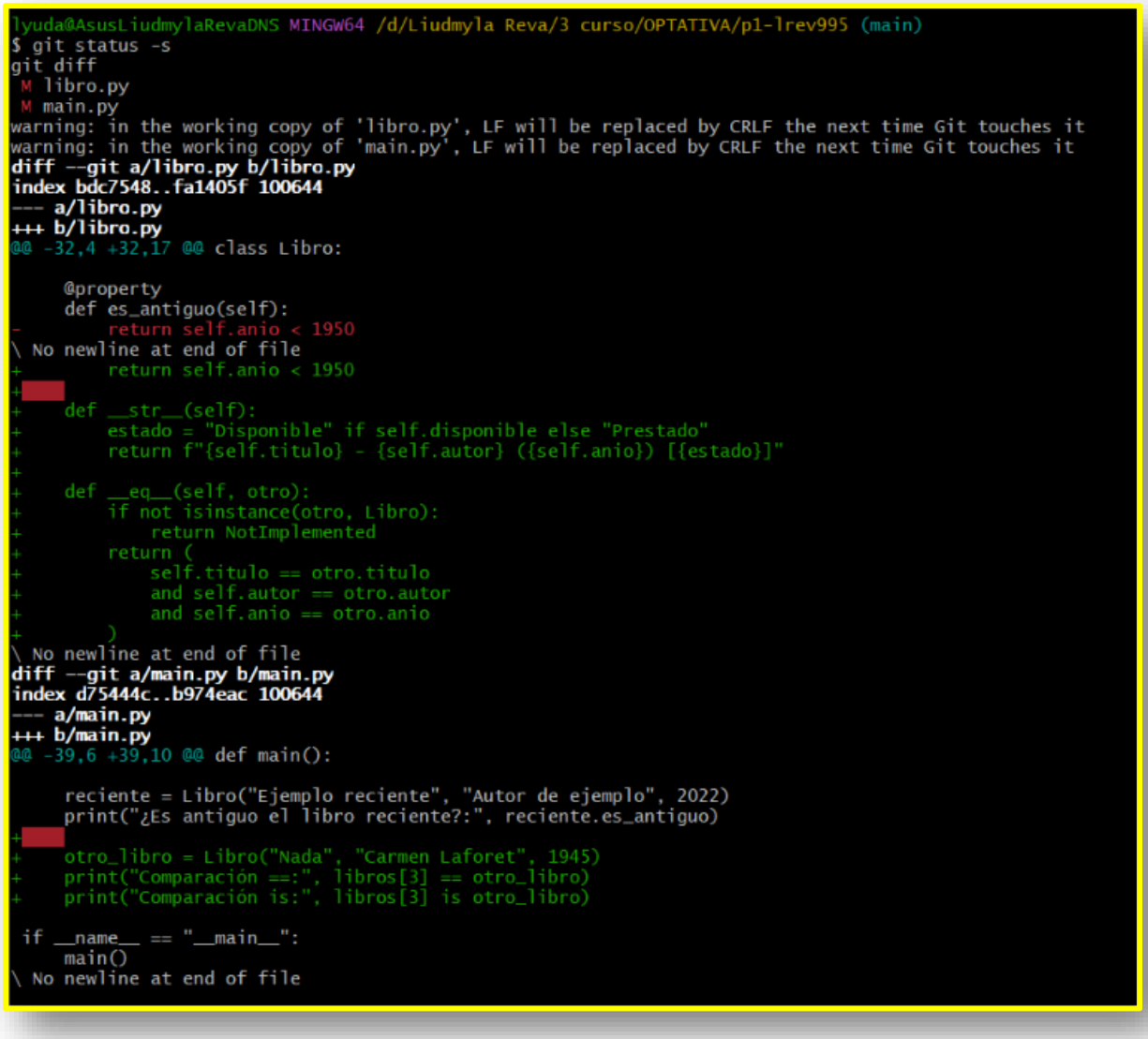
    print(libros[3] == otro_libro)  # True
    print(libros[3] is otro_libro)  # False
```

En Git: guardar y publicar v7

```
git status -s
git diff
```

Ejecuté `git status -s` para verificar el estado de los archivos tras implementar los métodos especiales, comprobando que tanto `libro.py` como `main.py` figuran como modificados (`M`).

Revisé los cambios en detalle con `git diff`: en `libro.py` sobrescribí `__str__()` para formatear la salida del libro y `__eq__()` para comparar instancias por sus atributos, mientras que en `main.py` añadí las comprobaciones con `==` e `is`.



```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git status -s
git diff
 M libro.py
 M main.py
warning: in the working copy of 'libro.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
diff --git a/libro.py b/libro.py
index bdc7548..fa1405f 100644
--- a/libro.py
+++ b/libro.py
@@ -32,4 +32,17 @@ class Libro:

     @property
     def es_antiguo(self):
-        return self.anio < 1950
\ No newline at end of file
+        return self.anio < 1950
+
+    def __str__(self):
+        estado = "Disponible" if self.disponible else "Prestado"
+        return f"{self.titulo} - {self.autor} ({self.anio}) [{estado}]"
+
+    def __eq__(self, otro):
+        if not isinstance(otro, Libro):
+            return NotImplemented
+        return (
+            self.titulo == otro.titulo
+            and self.autor == otro.autor
+            and self.anio == otro.anio
+        )
\ No newline at end of file
diff --git a/main.py b/main.py
index d75444c..b974eac 100644
--- a/main.py
+++ b/main.py
@@ -39,6 +39,10 @@ def main():

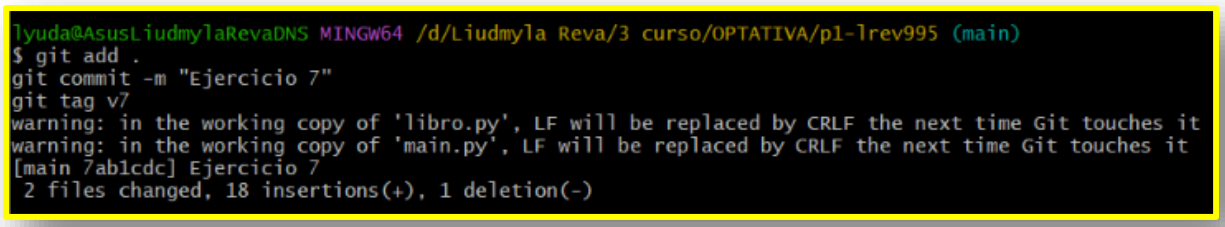
     reciente = Libro("Ejemplo reciente", "Autor de ejemplo", 2022)
     print("¿Es antiguo el libro reciente?:", reciente.es_antiguo)
+
+    otro_libro = Libro("Nada", "Carmen Laforet", 1945)
+    print("Comparación ==:", libros[3] == otro_libro)
+    print("Comparación is:", libros[3] is otro_libro)

 if __name__ == "__main__":
     main()
\ No newline at end of file
```

```
git add .
```

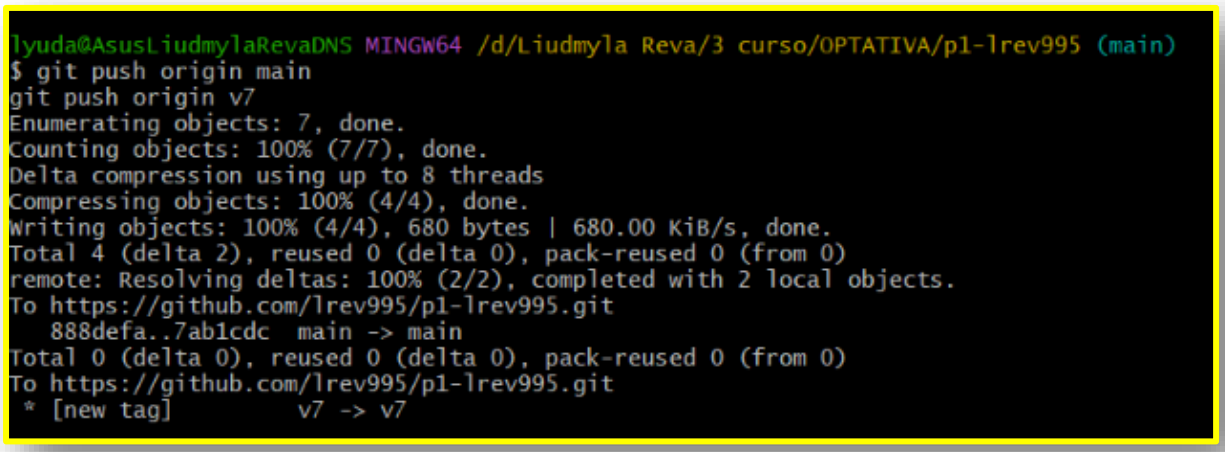
```
git commit -m "Ejercicio 7"
git tag v7
```

Preparé los cambios de los métodos especiales `__str__` y `__eq__` con `git add .`, registré el commit correspondiente al Ejercicio 7 (`7ab1cdc`) y creé la etiqueta `v7` para marcar esta versión en el repositorio.




```
git push origin main
git push origin v7
```

Subí los cambios del Ejercicio 7 a la rama remota con `git push origin main` y publiqué la etiqueta `v7` en GitHub mediante `git push origin v7`, dejando sincronizado el estado del proyecto.



Ejercicio 8. Funciones lambda, map() y filter()

Libros ordenados:

```
libros_ordenados = sorted(libros, key=lambda libro: libro.anio)

for libro in libros_ordenados:
```

```
        print(libro)
```

Libros disponibles:

```
    disponibles = filter(lambda libro: libro.disponible, libros)

    for libro in disponibles:
        print(libro)
```

Títulos de los libros disponibles:

```
        intentar_prestar(libros[1])

        titulos_disponibles = map(
            lambda libro: libro.titulo,
            filter(lambda libro: libro.disponible, libros)
        )

        for titulo in titulos_disponibles:
            print(titulo)
```

También se pueden obtener los títulos de los libros disponibles:

```
        titulos_disponibles = [
            libro.titulo for libro in libros if libro.disponible
        ]
```

Libros prestados:

```
        prestados = filter(lambda libro: not libro.disponible, libros)

        for libro in prestados:
            print(libro)
```

Títulos de los libros prestados:

```
        titulos_prestados = map(
            lambda libro: libro.titulo,
            filter(lambda libro: not libro.disponible, libros)
        )

        for titulo in titulos_prestados:
            print(titulo)
```

También se pueden obtener los libros prestados:

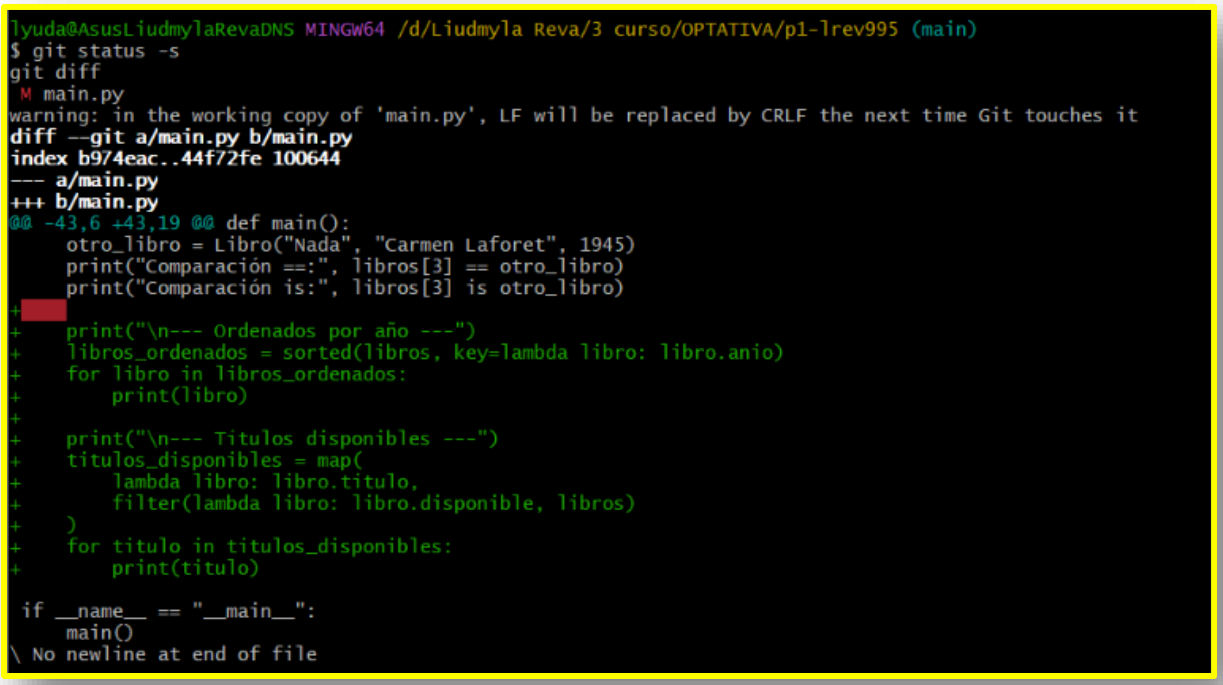
```
        titulos_no_disponibles = [
            libro.titulo for libro in libros if not libro.disponible
        ]
```

En Git: guardar y publicar v8

```
git status -s
git diff
```

Ejecuté `git status -s` para comprobar el estado del proyecto en el último ejercicio, verificando que únicamente el fichero `main.py` figura como modificado (`M`).

Revisé las modificaciones finales con `git diff`, confirmando la incorporación de expresiones `lambda` junto con `sorted()` para ordenar los libros por año, y la combinación de `filter()` y `map()` para mostrar los títulos disponibles.



```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git status -s
git diff
 M main.py
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
diff --git a/main.py b/main.py
index b974eac..44f72fe 100644
--- a/main.py
+++ b/main.py
@@ -43,6 +43,19 @@ def main():
     otro_libro = Libro("Nada", "Carmen Laforet", 1945)
     print("Comparación ==:", libros[3] == otro_libro)
     print("Comparación is:", libros[3] is otro_libro)
+
+    print("\n--- Ordenados por año ---")
+    libros_ordenados = sorted(libros, key=lambda libro: libro.anio)
+    for libro in libros_ordenados:
+        print(libro)
+
+    print("\n--- Titulos disponibles ---")
+    titulos_disponibles = map(
+        lambda libro: libro.titulo,
+        filter(lambda libro: libro.disponible, libros)
+    )
+    for titulo in titulos_disponibles:
+        print(titulo)

 if __name__ == "__main__":
     main()
\ No newline at end of file
```

```
git add .
git commit -m "Ejercicio 8"
git tag v8
```

Añadí las funciones de ordenación y filtrado al área de preparación con `git add .`, registré el commit final correspondiente al Ejercicio 8 (`701ceac`) y creé la etiqueta `v8` para dejar cerrada esta última versión en el repositorio local

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git add .
git commit -m "Ejercicio 8"
git tag v8
warning: in the working copy of 'main.py', LF will be replaced by CRLF the next time Git touches it
[main 701ceac] Ejercicio 8
 1 file changed, 13 insertions(+)
```

git push origin main
git push origin v8

Subí los últimos cambios del Ejercicio 8 a la rama remota con `git push origin main` y publiqué la etiqueta final `v8` en GitHub mediante `git push origin v8`, dejando todo el proyecto completamente actualizado y sincronizado en el repositorio remoto.

```
lyuda@AsusLiudmylaRevaDNS MINGW64 /d/Liudmyla Reva/3 curso/OPTATIVA/p1-lrev995 (main)
$ git push origin main
git push origin v8
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 480 bytes | 480.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/lrev995/p1-lrev995.git
   7ab1cdc..701ceac  main -> main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/lrev995/p1-lrev995.git
 * [new tag]         v8 -> v8
```

Comprobación final:

Historial

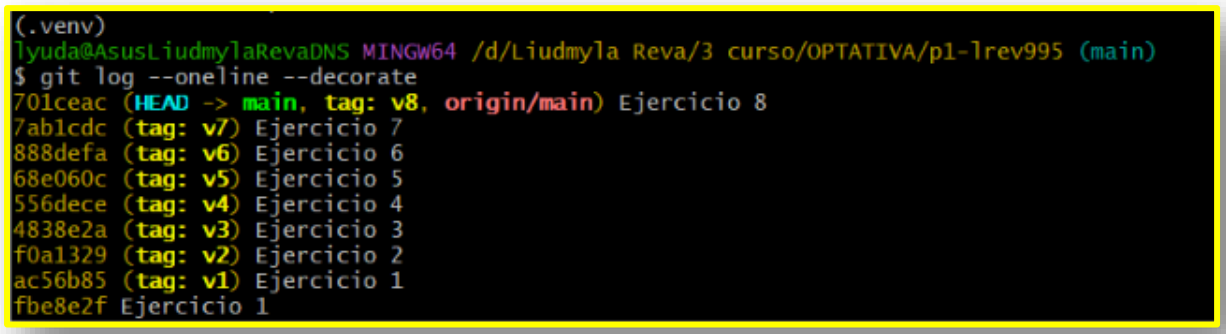
git log
git log --oneline --decorate

```
(.venv) PS C:\Users\pilia\Desktop\OPT\p1-maleram052> git log --oneline --decorate
28aac3c (HEAD -> main, tag: v1, p1-maleram052/main) Ejercicio1
(.venv) PS C:\Users\pilia\Desktop\OPT\p1-maleram052> git log
commit 28aac3c0f28ddf323049d1995c1a9c79b2565641 (HEAD -> main, tag: v1, p1-maleram052/main)
Author: Pilar Alegre <maleram052@g.educaand.es>
Date:   Wed Nov 11 12:31:26 2026 +0100

    Ejercicio1
(.venv) PS C:\Users\pilia\Desktop\OPT\p1-maleram052>
```

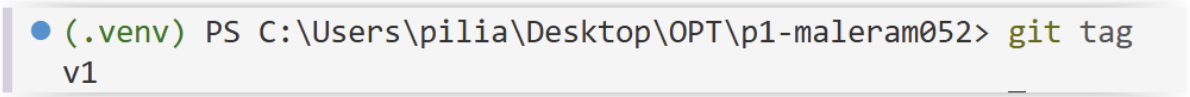
Comprobé el historial resumido del repositorio con `git log --oneline --decorate` teniendo el entorno virtual `(.venv)` activo,

verificando que todos los ejercicios desde `Ejercicio 1` hasta `Ejercicio 8` están registrados en orden con sus respectivas etiquetas (`v1` a `v8`) y sincronizados con la rama remota `origin/main`.

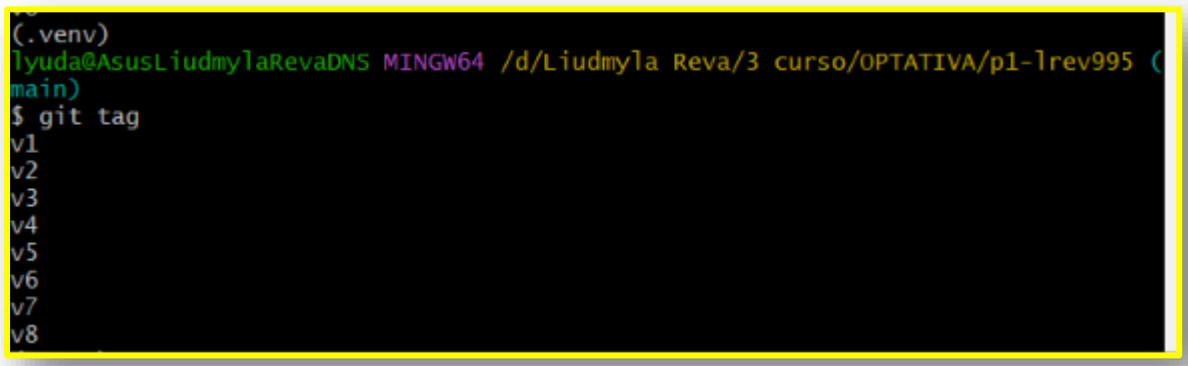


Etiquetas:

`git tag`



Listé las etiquetas creadas en el repositorio con `git tag`, confirmando que todas las versiones desde `v1` hasta `v8` están registradas correctamente en mi proyecto.

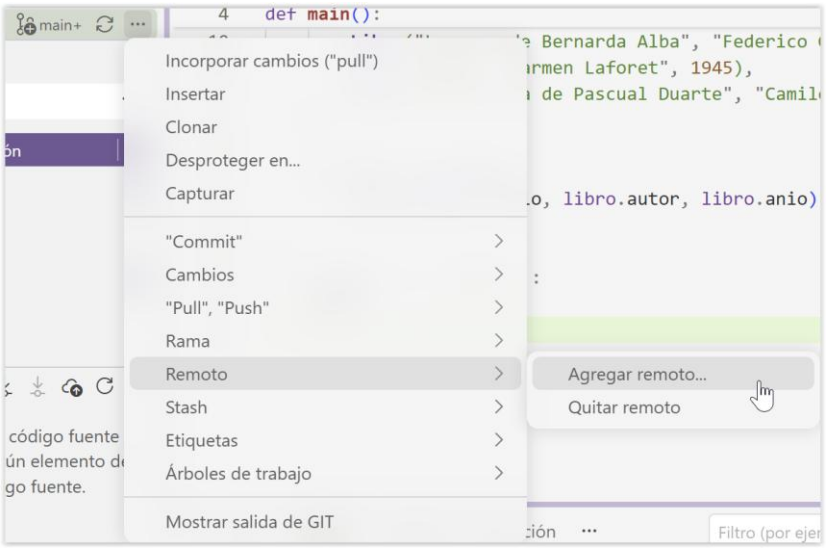


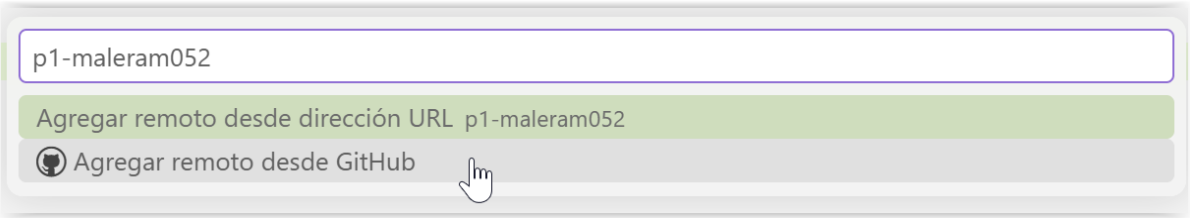
Utilizar los repositorios desde Visual Studio Code:

(Solo he añadido algunos pasos, los demás ya lo hemos visto anteriormente)

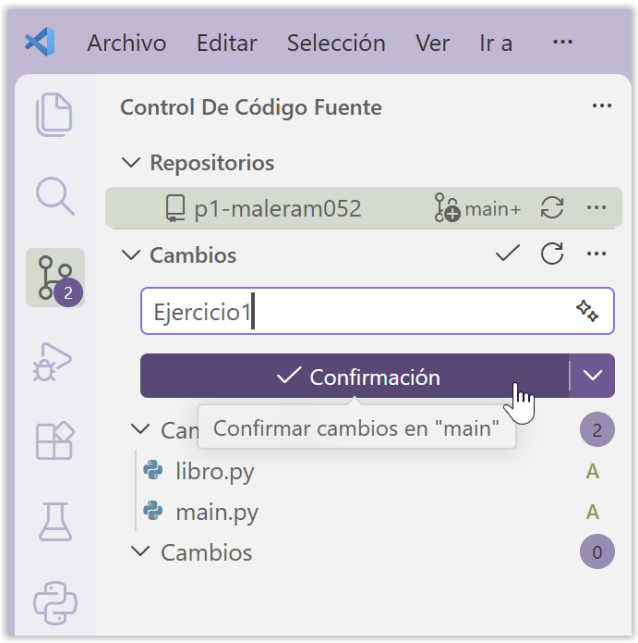
Agregar remoto:

`git add remote…`

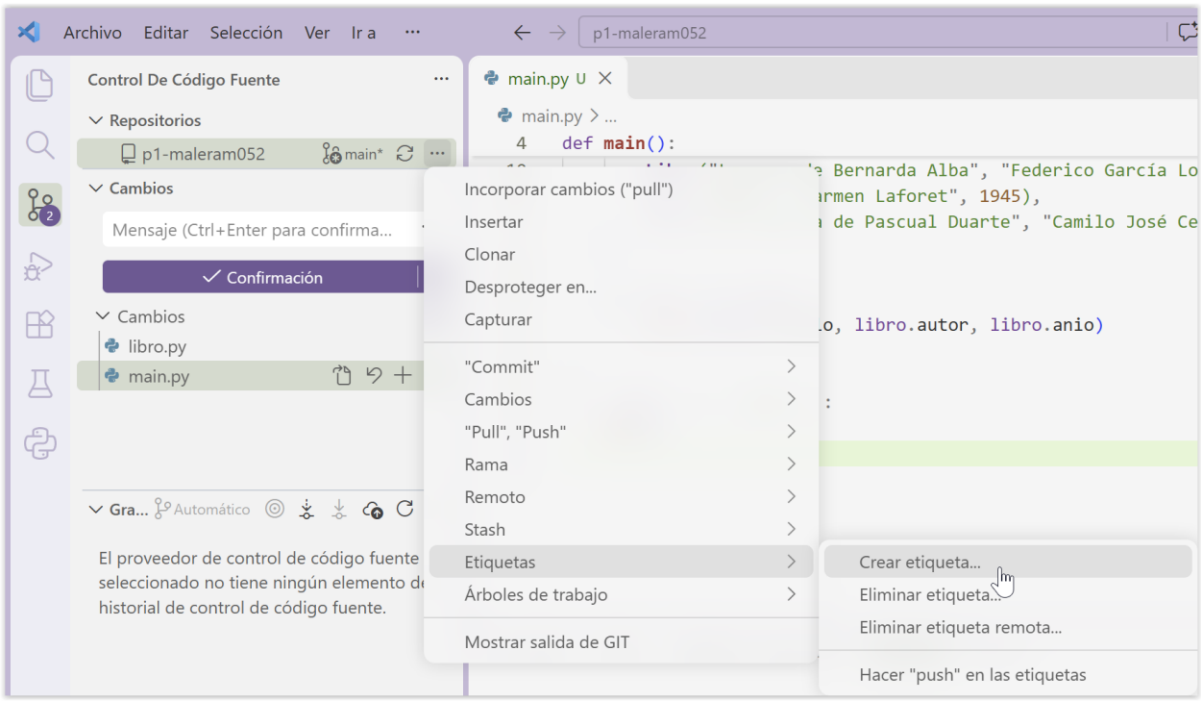




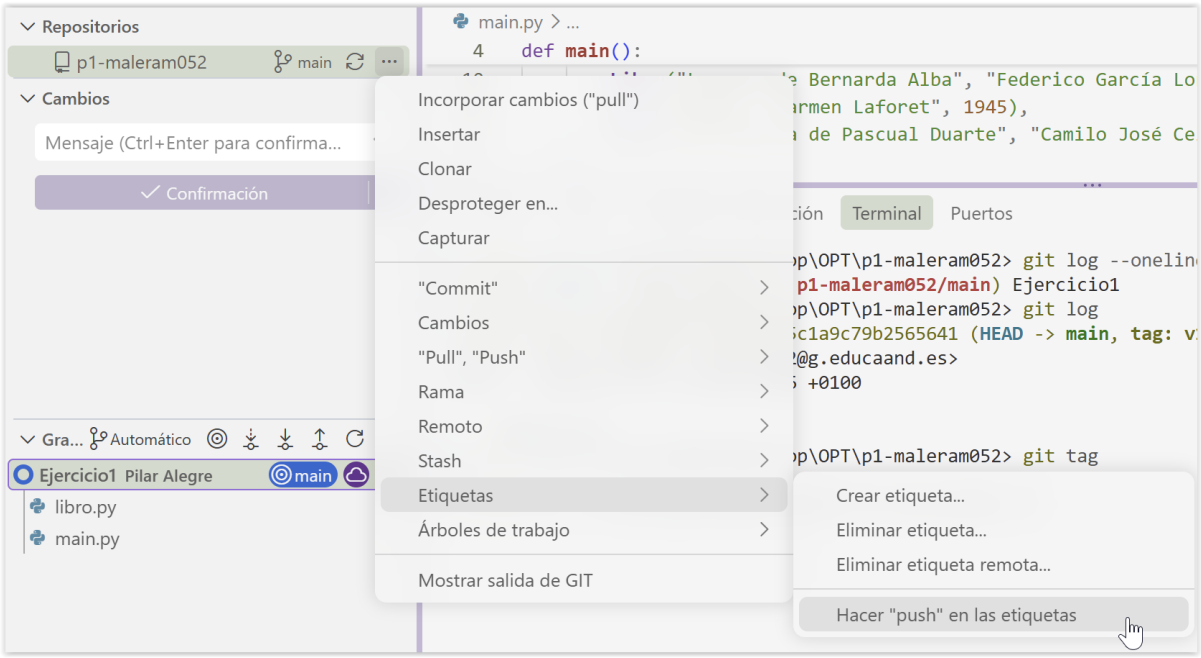
Commit:

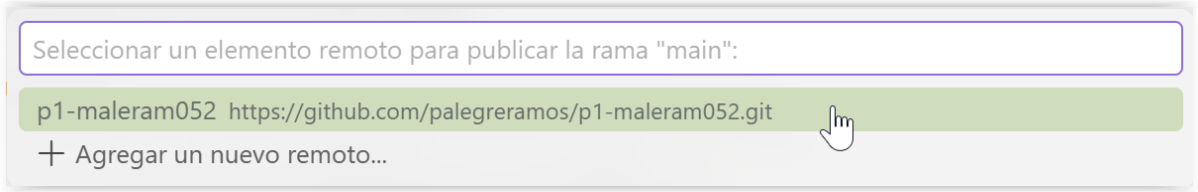


Crear y hacer push en etiqueta:


Archivo Editar Selección Ver Ir a
p1-maleram052
Control De Código Fuente
Repositorios
p1-maleram052 main*
Cambios
Mensaje (Ctrl+Enter para confirma...
Confirmación
Cambios
libro.py
main.py
Gra... Automático
El proveedor de control de código fuente seleccionado no tiene ningún elemento de historial de control de código fuente.
main.py U
main.py > ...
4 def main():
Incorporar cambios ("pull")
Insertar
Clonar
Desproteger en...
Capturar
"Commit"
Cambios
"Pull", "Push"
Rama
Remoto
Stash
Etiquetas
Árboles de trabajo
Mostrar salida de GIT
Crear etiqueta...
Eliminar etiqueta...
Eliminar etiqueta remota...
Hacer "push" en las etiquetas

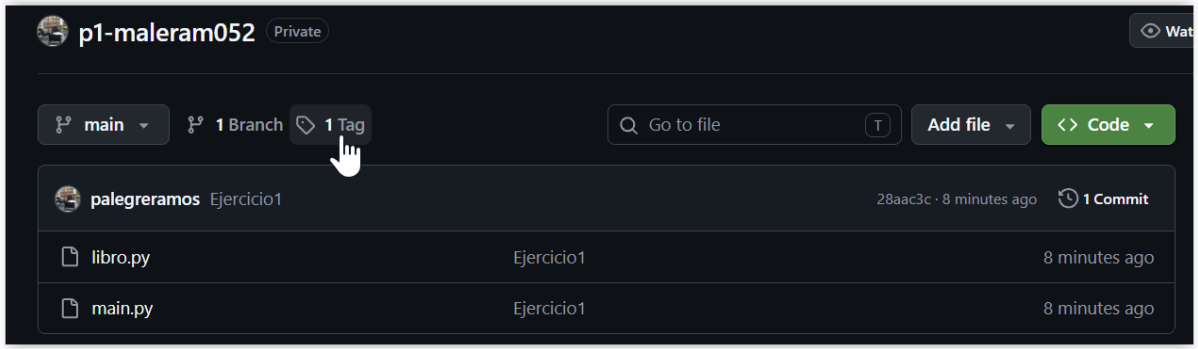

v1
Por favor proporcione un nombre para la etiqueta (Presione "Entrar" para confirmar o "Esc" para cancelar)

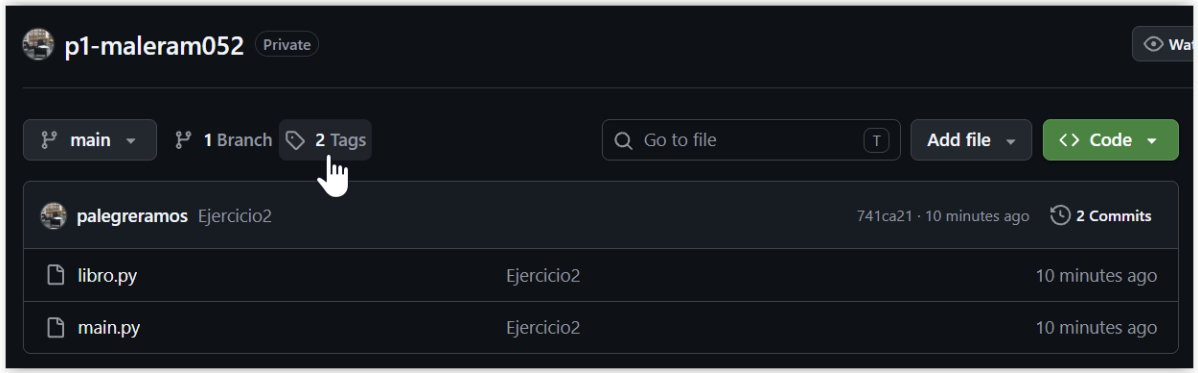

Repositorios
p1-maleram052 main
Cambios
Mensaje (Ctrl+Enter para confirma...
Confirmación
Gra... Automático
Ejercicio1 Pilar Alegre main
libro.py
main.py
main.py > ...
4 def main():
Incorporar cambios ("pull")
Insertar
Clonar
Desproteger en...
Capturar
"Commit"
Cambios
"Pull", "Push"
Rama
Remoto
Stash
Etiquetas
Árboles de trabajo
Mostrar salida de GIT
Terminal
Puertos
Crear etiqueta...
Eliminar etiqueta...
Eliminar etiqueta remota...
Hacer "push" en las etiquetas


Seleccionar un elemento remoto para publicar la rama "main":
p1-maleram052 https://github.com/palegreramos/p1-maleram052.git
Agregar un nuevo remoto...

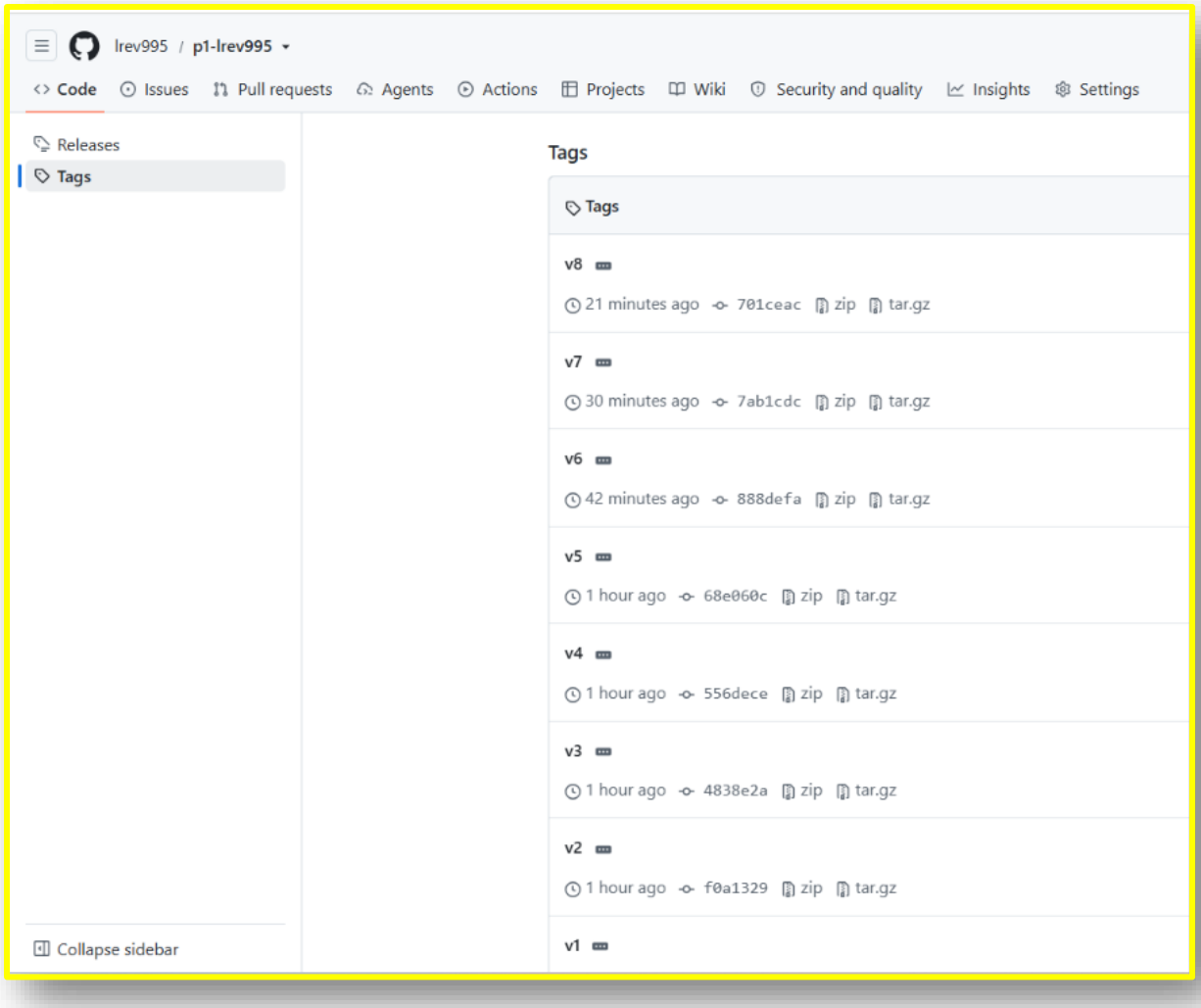


Ver las etiquetas en GitHub:

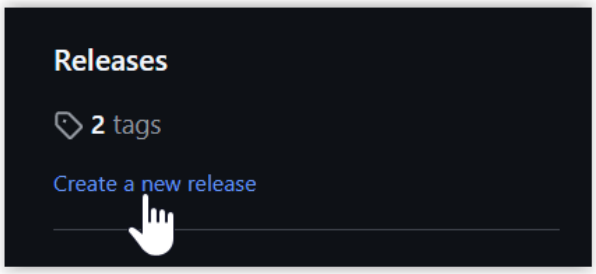




*Verifiqué la sección de **Tags** directamente en el repositorio remoto de GitHub (`lrev995/p1-lrev995`), confirmando que las ocho etiquetas (`v1` a `v8`) se han publicado correctamente y apuntan a cada uno de los commits correspondientes a los ejercicios de la práctica.*

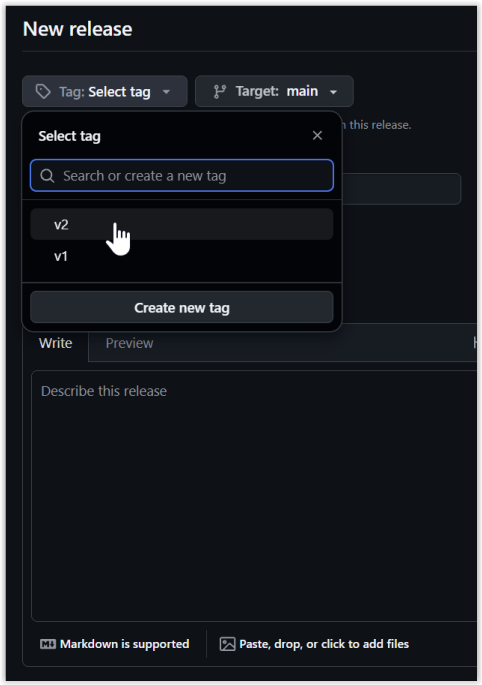


A partir de una etiqueta, se puede crear un release:



Commit: registra un cambio en el historial del proyecto.
Tag (etiqueta): identifica un commit concreto como un punto importante del desarrollo.
Versión: identifica un estado del software, normalmente mediante un número.
Release: es una publicación de una versión, por ejemplo en GitHub.
En proyectos reales se utiliza frecuentemente el versionado semántico, con el formato MAJOR.MINOR.PATCH.

Mayor (major)	1.0.0 → 2.0.0	Cambios importantes con respecto a versiones anteriores
Menor (minor)	1.0.0 → 1.1.0	Nuevas funcionalidades que no alteran el funcionamiento
Corrección (patch)	1.0.0 → 1.0.1	Corrección de errores



*Creé y publiqué la **Release** oficial `Versión 1.0.0 (v8) – Práctica Biblioteca` en GitHub a partir de la etiqueta `v8` (commit `701ceac`), marcándola como `Latest` y generando automáticamente los archivos descargables del código fuente en formatos `.zip` y `.tar.gz`.*



Plugin recomendado:
[autoDocstring - Python Docstring Generator - Visual Studio Marketplace]